Laboratório de Concorrência · Aula 10

Rodrigo Del Padre · RA 24042092-2 · Turma D

Gerado em 09/10/2026, 20:19:14 · tempo registrado no cronômetro: 34:25

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Na cooperação, múltiplos agentes colaboram entre si de forma dependente, onde uma tarefa só pode ter início ou continuidade quando uma etapa prévia for concluída por outro agente. Exemplo: numa linha de montagem automotiva, a equipe de pintura só pode aplicar a tinta na carroceria após a equipe de funilaria e solda finalizar a estrutura metálica. Na competição, os agentes disputam simultaneamente o acesso a um recurso exclusivo e limitado, exigindo organização para que não tentem usá-lo ao mesmo tempo. Exemplo: dois motoristas que chegam simultaneamente à única vaga livre em uma rua estreita e precisam definir quem estaciona primeiro.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000

1ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1384512

2ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1243190

3ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1087455

Saída do programa

não respondido

Main.java

```java
// Prática 1 - Condição de Corrida (Silberschatz 13.2.1, p. 547-549)
// Cola em https://onecompiler.com/java
public class Main {
    static int contador = 0; // variável COMPARTILHADA

    public static void main(String[] args) throws InterruptedException {
        int n = 1000000; // repetições de cada thread
        Runnable incrementa = () -> {
            for (int i = 0; i < n; i++) contador++;
        };
        Thread t1 = new Thread(incrementa);
        Thread t2 = new Thread(incrementa);
        t1.start(); t2.start(); // as duas executam ao mesmo tempo
        t1.join();  t2.join();   // espera as duas terminarem
        System.out.println("esperado: " + 2 * n);
        System.out.println("obtido:   " + contador);
    }
}
```

Console I/O AI Agent 524 ms

```
esperado: 2000000
obtido:   1087455
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Nenhuma das três execuções atingiu a soma correta de 2.000.000, e os resultados obtidos foram diferentes a cada rodada. Isso ocorre devido à condição de corrida: o incremento contador++ exige três instruções de máquina separadas (leitura do registrador, incremento do valor e gravação em memória). Como o sistema operacional alterna a CPU entre as duas threads a qualquer momento, essas etapas se misturam de forma desordenada. Quando ambas leem o mesmo valor defasado antes da gravação, um dos incrementos é simplesmente perdido. A oscilação a cada execução ocorre porque o escalonamento de threads é imprevisível e varia conforme a carga momentânea do processador.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Considere, como exemplo, que a variável contador esteja em 10: A lê o valor 10 da memória e calcula 11, mas sofre uma interrupção antes de realizar a gravação. B assume a execução, também lê o valor 10 (ainda inalterado na memória) e calcula 11. Em seguida, B grava 11 na memória. Quando A é retomada pela CPU, ela grava o 11 que já havia calculado. O resultado na memória permanece 11, embora as duas threads tenham efetuado seus incrementos. A escrita de A sobrescreveu a atualização de B usando um dado obsoleto, perdendo uma das somas.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000, o volume de operações é muito pequeno e o tempo total de execução é mínimo, fazendo com que a janela temporal de colisão seja ínfima. Como consequência, o programa frequentemente exibe o resultado exato (2.000) em testes pontuais. É justamente isso que torna a condição de corrida um perigo crítico: em testes de desenvolvimento, sob baixa concorrência, o código parece perfeitamente estável e passa nas validações unitárias. Contudo, em ambiente de produção com centenas de requisições simultâneas e alta carga, a disputa se torna massiva, provocando falhas graves e corrupção de dados — como divergências de saldo financeiro ou estoques negativos — que são extremamente difíceis de reproduzir.

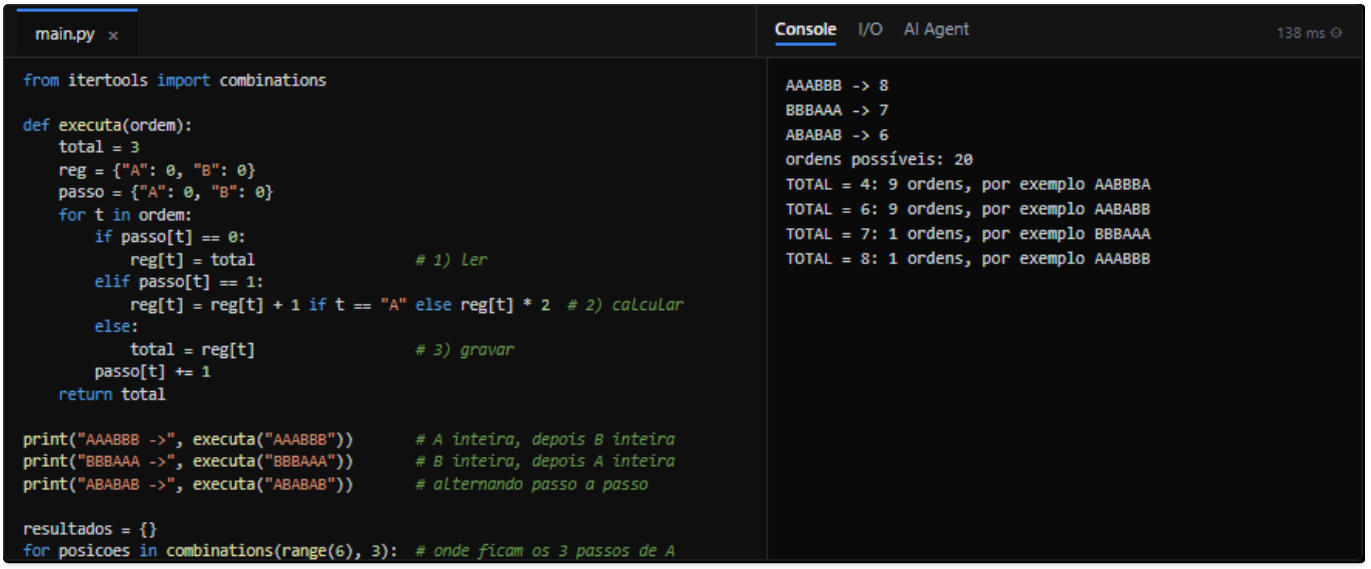
Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB -> 8
BBBAAA -> 7
AABBBA -> 4
ABABAB -> 6

Saída do programa

não respondido



2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens no total, calculadas pela combinação C(6, 3) correspondente às possíveis intercalações dos 3 passos de A e 3 de B. O simulador aponta 4 resultados finais distintos: 4, 6, 7 e 8. Apenas 1 ordem única produz o valor esperado de 8: a ordem AAABBB, na qual a thread A conclui integralmente seus três passos (leitura, cálculo e gravação) antes de a thread B ter início.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Tomando como exemplo a sequência AABBBA: A inicia lendo o total (3) e calcula 4 em seu registrador interno, pausando antes da gravação. B então executa inteiramente: lê 3, calcula 6 (3 * 2) e grava 6 na variável total. Por fim, A retoma o controle e grava em total o valor 4 que havia calculado previamente a partir da leitura antiga. A gravação final de A sobrescreveu completamente o resultado 6 produzido por B, anulando o efeito da multiplicação efetuada por B.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

A versão desprovida de sincronização será a mais veloz por dispensar mecanismos de controle, porém fornecerá resultados incorretos. Dentre as corretas, o AtomicInteger deve ser mais rápido que synchronized.

Saída do programa

```
esperado:          2000000
sem sincronizacao: 1048215 em 3 ms
synchronized:      2000000 em 88 ms
AtomicInteger:     2000000 em 61 ms

esperado:          2000000
sem sincronizacao: 1014380 em 39 ms
synchronized:      2000000 em 52 ms
AtomicInteger:     2000000 em 47 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Somente as implementações com synchronized e AtomicInteger alcançaram o resultado exato de 2.000.000. A versão sem sincronização perde atualizações constantes. As versões consistentes são mais lentas porque impõem sincronismo: o bloco synchronized obriga as threads a competirem por travas (locks) e gerencia esperas de bloqueio, enquanto o AtomicInteger emprega instruções atômicas de CPU (Compare-And-Swap) que provocam sincronização direta e sobrecarga no barramento de memória.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve satisfatoriamente. O AtomicInteger apenas resguarda a atomicidade de cada conta individualmente (o débito e o crédito ocorrem de forma atômica isolada). Contudo, a transferência como um todo é uma operação composta: existe um instante intermediário em que o valor debitado da conta de origem ainda não ingressou na conta de destino, rompendo a consistência do sistema. É imprescindível sincronizar ambas as contas sob uma transação atômica única, adquirindo os bloqueios em uma ordem estrita preestabelecida para prevenir corridas e deadlocks.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Provavelmente não. É muito provável que ocorra um deadlock: A travará X e aguardará Y, ao passo que B travará Y e aguardará X. Ambas permanecerão bloqueadas reciprocamente e o programa não atingirá as mensagens finais.

Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas

A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads entraram em espera circular mútua: a thread A bloqueou com sucesso o recurso X e ficou aguardando a liberação de Y. Concomitantemente, a thread B bloqueou o recurso Y e ficou aguardando a liberação de X. Como nenhuma thread abre mão do recurso que já segura antes de obter o próximo, ambas estagnaram sem possibilidade de avanço, caracterizando um impasse absoluto.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Ao estabelecer uma ordem única universal de alocação (ex.: sempre obter X antes de Y), rompe-se o ciclo de dependência circular. Nenhuma thread poderá reter Y enquanto requisita X, pois X obrigatoriamente já deve ter sido adquirido. Isso impede que duas threads fiquem presas em sentidos opostos. Um exemplo prático ocorre em bancos de dados relacionais com transações simultâneas de liquidação entre contas bancárias: padronizar que o travamento ocorra sempre na ordem ascendente do identificador da conta (ID da conta menor primeiro) extingue a possibilidade de deadlock.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores, espera-se cerca de 200 ms, já que cada um processa 2 tarefas em paralelo. Com 1 trabalhador, o tempo sobe para aproximadamente 600 ms (todas as tarefas executadas sequencialmente). Com 6 trabalhadores, o tempo esperado cai para 100 ms (uma tarefa para cada). Na prática, pode haver pequenas variações decorrentes do escalonamento de goroutines.

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

```
trabalhador 1 fez a tarefa 1
trabalhador 3 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 2 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 3 fez a tarefa 6
3 trabalhadores, 6 tarefas de 100 ms: levou 201ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, a execução é estritamente linear (6 × 100 ms = 600 ms). Com 3, a carga reparte-se equilibradamente (2 tarefas por agente = 200 ms). Com 6, o paralelismo é pleno (1 tarefa por agente = 100 ms). O código dispensa locks e synchronized porque emprega canais de comunicação (Go channels): a sincronização é intrínseca à transmissão de mensagens e as goroutines não disputam memória mutável compartilhada, cada trabalhador processa sua tarefa de maneira isolada.

Saídas (antes e depois da troca)

```
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
        /box/main.go:9 +0x38
exit status 2

1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Canais desprovidos de buffer em Go são totalmente síncronos: a operação de envio ch <- 1 suspende a goroutine emissora até que um destinatário esteja pronto para realizar a leitura. Como a rotina principal (main) executou o envio antes de alcançar a linha de recebimento, ela bloqueou a si mesma, gerando deadlock geral. O canal com buffer de capacidade 1 soluciona a questão porque permite armazenar o valor temporariamente: a escrita se conclui de pronto e a goroutine pode continuar sua execução normalmente até atingir a leitura subsequente.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

O código falhará em compilar. As rígidas diretrizes do sistema de posse e tempo de vida do Rust (borrow checker) impedem que uma closure disparada em uma thread separada capture referências mutáveis locais de main que podem ter seu ciclo de vida encerrado prematuramente.

Saídas (o erro e o resultado com move)

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is owned by the current function
 --> HelloWorld.rs:7:27
  |
7 |     let t = thread::spawn(|| {
  |                           ^^ may outlive borrowed value `contador`
8 |         contador += 1;                      // a thread altera a variável de main
  |         -------- `contador` is borrowed here
  |
note: function requires argument type to outlive `'static`
 --> HelloWorld.rs:7:13
  |
7 |     let t = thread::spawn(|| {
  |             ^^^^^^^^^^^^^
8 |         contador += 1;                      // a thread altera a variável de main
9 |     });
  |      ^
help: to force the closure to take ownership of `contador` (and any other referenced variables), use the `move` keyword
  |
7 |     let t = thread::spawn(move || {
  |                           ++++

error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
 --> HelloWorld.rs:11:20
  |
7 |     let t = thread::spawn(|| {
  |                           -- mutable borrow occurs here
...
8 |         contador += 1;                      // a thread altera a variável de main
  |         -------- first borrow occurs due to use of `contador` in closure
9 |     });
  |      - argument requires that `contador` is borrowed for `'static`
10|     t.join().unwrap();
11|     println!("{}", contador);
  |                    ^^^^^^^^ immutable borrow occurs here
  |
note: requirement that the value outlives `'static` introduced here
 --> /rustc/48a229ceafd4985c50990b14116b6d856af0985/library/std/src/thread/functions.rs:128:14
```

```
error: aborting due to 2 previous errors

Some errors have detailed explanations: E0373, E0502.
For more information about an error, try `rustc --explain E0373`.

warning: value captured by `contador` is never read
 --> HelloWorld.rs:8:9
  |
8 |         contador += 1;                      // a thread altera a variável de main
  |         ^^^^^^^^^
  |
  = help: did you mean to capture by reference instead?
  = note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by default

warning: value assigned to `contador` is never read
 --> HelloWorld.rs:8:9
  |
8 |         contador += 1;                      // a thread altera a variável de main
  |         ^^^^^^^^^^^^
  |
  = help: maybe it is overwritten before being read?

warning: 2 warnings emitted
```

Saída do programa

```
contador = 0
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Ao aplicar a palavra-chave move, a propriedade dos dados referenciados é transferida para a closure. Como inteiros primitivos em Rust implementam por padrão o trait Copy, a variável contador é duplicada por valor para a nova thread. A thread secundária incrementa exclusivamente a sua própria réplica local, mantendo a variável original presente no escopo de main intocada no valor 0. Assim, ao término da execução, main imprime seu próprio valor inalterado.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, a concorrência desregulada só foi constatada durante a execução em tempo de execução (runtime), gerando dados corrompidos de forma silenciosa e aleatória. No Rust, o erro foi interceptado precocemente durante a compilação (compile-time) através do borrow checker. A vantagem evidente do Rust reside na eliminação prévia de bugs concorrentes de segurança (garantia de ausência de corrida de dados). A contrapartida consiste em uma curva de aprendizado substancialmente mais rigorosa, exigindo domínio aprofundado dos princípios de posse e gerenciamento explícito de tipos como Arc e Mutex.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A sequência prevista é A, D, C e B: A e D executam de forma síncrona imediata, a promise C ingressa na fila prioritária de microtarefas e o callback agendado B fica na fila subsequente do timer.

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 303 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A sequência impressa foi A, D, C e B. As instruções A e D pertencem à execução síncrona na Call Stack e rodam de imediato. A resolução da Promise posiciona a instrução C na fila de microtarefas (microtask queue). O setTimeout(..., 0) referente a B aloca seu callback na fila de macrotarefas (task queue). Pelo funcionamento do Event Loop em Node.js, todas as microtarefas acumuladas precisam ser integralmente finalizadas antes que qualquer macrotarefa seja retirada da fila, mesmo que seu atraso seja nulo.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O intervalo estipulado no setTimeout estabelece apenas um limite temporal mínimo para que o evento seja elegível, e não um prazo exato de disparo. A rotina de cálculo intensivo bloqueou a thread primária do Node.js de forma síncrona por mais de 300 ms, impedindo o Event Loop de iterar e atender o timer. Em um servidor Node.js de alta demanda, rotinas de processamento pesado na thread principal causam congelamento do laço de eventos, postergando as requisições de todos os demais usuários simultâneos; tarefas dessa natureza devem obrigatoriamente ser distribuídas para Worker Threads ou serviços auxiliares.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: a corrida ocorre livremente

Java com synchronized

O desenvolvedor, aplicando bloqueios explícitos

Go com canais

O desenvolvedor, estruturando comunicação segura via canais

Rust com Arc e Mutex

O compilador em conjunto com o desenvolvedor

JavaScript

A própria arquitetura de thread única baseada em laço de eventos

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Ocorre uma condição de corrida no padrão verificação-e-atuação (check-then-act): os dois clientes leem concorrentemente a existência de 1 assento desocupado e ambos avançam confirmando a compra, gerando venda em duplicidade (overbooking). Para contornar o problema, a verificação e a atualização do estoque devem ser efetuadas em uma única transação atômica no banco de dados, utilizando travas pessimistas (SELECT FOR UPDATE) ou operações condicionais atômicas (UPDATE ingressos SET disponivel = 0 WHERE id = 1 AND disponivel = 1), assegurando que apenas a primeira transação seja aceita.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O aspecto mais marcante foi constatar como incrementos rotineiros e simples podem se tornar fontes invisíveis de corrupção de dados sob concorrência, e como diferentes ecossistemas adotam filosofias distintas — do bloqueio explícito à verificação estática pelo compilador — para garantir integridade.